

PARALLEL COMPUTING WITH OPENMP

4.1 Introduction to OpenMP

4.2 Loop-level parallelism

4.2.1 for loops

4.3 Parallel regions

4.4 Synchronization

4.4.1 Critical and Atomic statements

Thus far in the course we have considered programming parallel machines using a distributed memory model, where several processes communicate through message passing, using the MPI library. This has been the traditional programming model used in the HPC community since distributed memory machines became popular during the late eighties. One of the main benefits of this programming model is that it can be used on all hardware, even machines that actually have a shared memory architecture. However, it has one major drawback; parallelizing a code typically requires substantial changes to the serial version.

In these notes we consider programming shared memory machines using a shared memory programming model. In particular we consider parallelization using the *OpenMP* standard. In later years, technology have reached a point where making a single processing core much faster is very challenging. In order to keep up

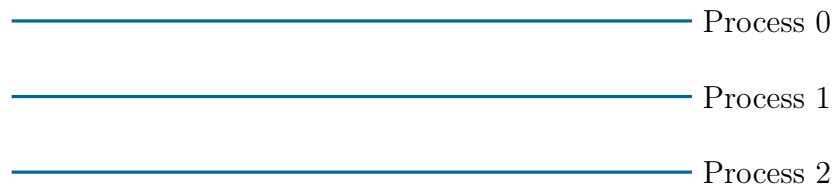


Figure 4.1: An illustration of the MPI programming model. We have several independent processes, and each of these processes have their own separate program flow.

with Moore's law for the next decades, vendors have started integrating several processing cores in one chip, even in processors targeted at desktop computers. At the moment, dual core processors are the norm on the desktop, with quad and hexacore processors being available in the high end market. All major vendors have signaled that they expect 8-16 cores to be the norm within a few years, and a few hundreds within a decade. This means that even for desktop class programs, programmers need to start developing parallel programs to utilize the available computing resources. The effect of this is a substantially increased interest in developing programming tools which allows for code parallization on shared memory architectures with little effort. While the OpenMP standard, which originated in the late ninties, initially was aimed at HPC applications, it has now been adapted much more broadly. The effort to improve the standard for the desktop market has also largely benefited the HPC community. The increased activity has resulted in two major revisions of the standard in the last five years, making it quite extensive. The scope of this document is not to give a thorough introduction to the whole API, but rather to give an idea of the main principles, as well as what new challenges and benefits programming using OpenMP offers compared to the traditional message passing tools.

4.5 The OpenMP programming model

OpenMP is a C/Fortran language extension for programming shared memory parallel machines. It is implemented and supported by most major vendors, including Intel, AMD, IBM and Oracle (SUN). It is also available in open source compilers such as GCC or Clang, as well as in professional versions of Microsoft Visual Studio.

It is built on the threading paradigm in combination with a parallel section view of the code. This means that the main program flow happens on one processor only, which is quite a difference from the MPI programming model. Consider

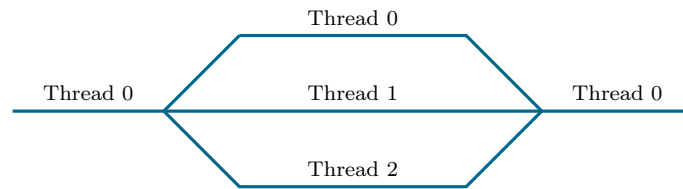


Figure 4.2: An illustration of the OpenMP programming model. We only have a single process, hence the main program flow only happens on a single processor. In sections of the code that we have marked as parallel, the program forks into multiple threads which work independently. At the end of the parallel section, these threads join together again and the program flow is returned to the first processor.

[Figure 4.1](#), which is an illustration of the MPI programming model. Here we have several processes, and each of these processes have their own separate program flow. That is, each process is a separate instance of the program. Synchronization of these processes is typically handled implicitly by using blocking communication calls, i.e. if you call `MPI_Send` in one process, this process halts until it receives a confirmation that the message has been processed. Likewise, on the receiver end, the process blocks the program flow in the `MPI_Recv` call until it has received the expected message.

In the OpenMP programming model, this is different. Consider [Figure 4.2](#) which is an illustration of this programming model. OpenMP is based on a fork/join programming model, where we only have a single instance of the program, i.e. the main program flow only happens on one processor. We mark certain sections of the program as being suitable for parallelization. When the program flow enters these sections of the code, the program *forks* into several threads which work independently of each other. At the end of the code sections the threads *join* with the thread running on processor 0 and the program flow is returned to this single processor. Thus in some sense one might say that the MPI programming model is embedded in the OpenMP model, but only in those parts of the code we have marked as parallel sections. This is only half the truth though. In MPI each process have their own *private* resources. Here however, this is not true. The threads within the parallel sections all have access to the *same* resources. This is crucial to keep in mind when designating the parallel sections of the code. One of the more common pitfalls is several threads trying to write to the same memory location, often due to using shared buffers during the calculations. It is thus highly recommended that you try to design your parallel sections in such a way that each thread has its own separate working buffers.

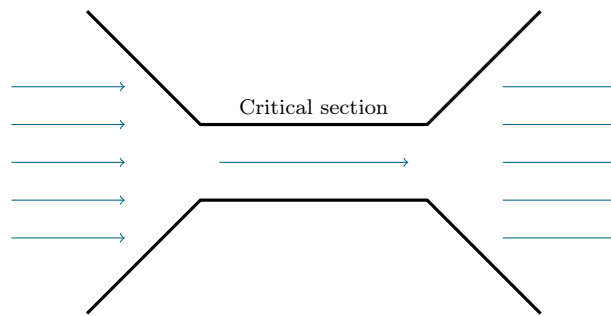


Figure 4.3: Illustration of a critical section. The incoming arrows represents the threads. All the threads are running concurrently, until they need to enter the critical section. Since only one thread can be inside the critical section at any time, the threads which want to enter need to wait until they obtain the key to the mutual exclusion lock, and hence their turn to enter the code section.

4.5.1 Critical section

If for some reason you cannot avoid several threads needing write access to the same resources, your only choice is to construct a critical section in your code to protect these resources. Consider [Figure 4.3](#). A critical section of the code is a section of the program in which only a single thread can be at any point in time. We construct such sections of the code using a tool known as a *mutual exclusion lock*, commonly referred to as a *mutex*. We make a section of the code critical by embedding it in a lock/unlock procedure. Prior to entering the critical section of the code, the thread requests a lock of the mutex. If the mutex is open when this is requested, the mutex is locked, and info about which thread has the key is recorded. This mutex is now locked until the thread associated with the key requests an unlock. The thread then moves on executing the critical section of the code. Upon completion of the critical section the thread unlocks the mutex and continues doing whatever we have told it to do next. Now, if a second thread attempts to lock the mutex while it still is locked, the second thread will stall in the lock call until it is able to obtain the key. Since the mutex only has a single key, it would only be able to obtain this key after the mutex has been unlocked by the thread which is currently holding it. Since this unlocking only happens after the critical section has been executed, we can then guarantee that only a single thread is within the critical section of the code at any time. We stress that this is something you should only use if there is no way around it, since the use of a mutex leads to a *serialization* of the critical section code. This can be catastrophic for the parallel performance of your code if a large part of the computation time is spent within such critical sections. Since this is meant to be a brief introduction, we will not discuss these issues further in the following.

4.6 How to use OpenMP

The idea behind the OpenMP API is that we give the compiler instructions on which sections of the code we want to be parallelized. This means that, in contrast to MPI, which works with all compilers, OpenMP requires specific support in the compiler. The compiler then handles work division between the available number of threads. This is in stark contrast to MPI where work division is something the programmer always have to decide up front, before modifying the serial code accordingly.

These instructions to the compiler are known as *pragma* commands. There are mainly two classes of OpenMP pragmas. The first class of pragmas are those which can be used in combination with loop constructs such as *for* loops. This is the most useful case in context of HPC, since the programs typically consist of multiple loops which apply the same operation to large datasets. Consider the serial snippet

```
for (int i=0; i<100; i++)  
    DoSomething(i);
```

or the equivalent in Fortran

```
do i = 1,100  
    call DoSomething(i)  
end do
```

For simplicity, we here assume that `DoSomething(i)` does not depend on any global resources, such as temporary working buffers. Hence this loop is highly suitable for parallelization using OpenMP. In addition we first assume that `DoSomething(i)` has a constant cost. To divide this loop among several threads, we simply do

```
#pragma omp parallel for schedule(static)  
for (int i=0; i<100; i++)  
    DoSomething(i);
```

Here we have our first example of an OpenMP directive. The pragma can be broken down into three parts.

`#pragma omp` all OpenMP directives start with this.

`parallel for` instructs the compiler that we want the following `for`-construct parallelized.

`schedule(static)` instructs the compiler to hand each thread approximately the same number of loop iterations up front. This is a good solution here since (we have assumed that) each call to `DoSomething(i)` has the same cost. Hence such a simple division will give good load balancing between the threads.

The ingredients in the Fortran version is the same, but the syntax is slightly different.

```
!OMP DO SCHEDULE(STATIC)
do i = 1,100
    call DoSomething(i)
end do
!OMP END DO
```

To avoid having to restate everything twice, we only give C examples in the following.

We can also run into situations where each call to `DoSomething(i)` has a different cost. One example where you can run into this scenario is if `DoSomething(i)` consists of an iterative method such as conjugate gradients. Each solution process can have a different solution time. In this case, if we give each thread a fixed number of loop iterations up front, we end up with poor load balancing between the threads. Fortunately OpenMP offers a mechanism to handle these situations. We simply do

```
#pragma omp parallel for schedule(dynamic)
for (int i=0; i<100; i++)
    DoSomething(i);
```

The only difference is the change of the schedule parameter in the pragma from static to dynamic. We here instruct the compiler to use a dynamic workload division between the threads. This means that we reserve one thread as a book-keeper/negotiator. Within this parallel section this thread has a simple task; keep track of which loop iterations have been performed and hand out a new one to a thread when it requests it. Initially each thread is given a single loop iteration to perform. Once the thread finishes this, it asks the negotiator thread for a new one. The threads keep doing this until all work has been performed.

If `DoSomething(i)` is fairly costly, this works very well. However, in some cases each `DoSomething(i)` may be rather cheap. In this case the cost of asking the negotiator for a new loop iteration between every calculation may dominate the actual computation time. OpenMP also offers a mechanism to try to minimize this problem. Instead of having the negotiator hand out a single loop iteration

when a thread asks for more work, it can hand out loop iterations in chunks. Consider

```
#pragma omp parallel for schedule(dynamic,5)
for (int i=0; i<100; i++)
    DoSomething(i);
```

The second parameter in the schedule (5) is the *chunk size*. This is the number of loop iterations a thread is (at most) given when it requests more work from the negotiator thread. Thus we can limit the number of times a thread has to communicate with the negotiator, hopefully making this part of the process less dominating.

OpenMP also offers a third scheduling mode, called *guided*. Consider

```
#pragma omp parallel for schedule(guided,5)
for (int i=0; i<100; ++i)
    DoSomething(i);
```

This is essentially a variant of dynamic scheduling, where we start out with a large chunk size. The chunks are then exponentially decreased until we reach a minimum, as specified in the chunk size. The idea here is that allocating large chunks initially is good for performance, since these will typically overlap fairly well. However, when the number of loop iterations left is small, we may end up in a situation where all the remaining loop iterations are allocated to a single thread. This is bad for performance since the other threads would then be left idle. By using progressively smaller chunks, the chance of this happening is reduced.

The second class of OpenMP directives are not tied to loop constructs. Instead they are to be used if we have sections of the code which are completely independent of each other. Consider the snippet

```
DoJob1();
DoJob2();
DoJob3();
```

If we are certain that these jobs are independent of each other, we can tell the compiler this fact, and ask for the different *sections* to be executed in parallel on several threads. We do

```
#pragma omp parallel sections
{
#pragma omp parallel section
{
    DoJob1();
}
```

```

    }
#pragma omp parallel section
    {
        DoJob2();
    }
#pragma omp parallel section
    {
        DoJob3();
    }
}

```

Here each section of the code would be performed in a separate thread, before the program flow again returns to processor 0 once all sections have been completed. This directive is not as useful as those used in combination with loop constructs, in particular we cannot as easily utilize a large number of threads. The reason for this is fairly straight forward. In this example we can at most use three threads, since there are only three sections of code specified. It is often hard to find a large number of independent code sections to allow for a larger number of threads. Large, expensive loops, however, are typically present in most codes.

4.7 π — OpenMP style

We have previously calculated π in both serial and MPI codes. We can certainly use OpenMP for this as well. In the original serial code we have a loop

```

double integrate(double x0, double x1, int n,
                 function_t f)
{
    double h = (x1-x0)/n;
    double result = 0.f;
    for (int i=0; i<n; i++) {
        double x = x0 + (i+.5f)*h;
        result += h*f(x);
    }

    return result;
}

```

This is the loop where the main work happens, and is what we should focus our effort on. In this case, it is embarassingly simple to parallelize the loop since there are no dependencies between the loop iterations. We can simply hand a number

of iterations to each thread, and then sum up the results afterwards. OpenMP makes this convenient through the reduction directive

```
double integrate(double x0, double x1, int n, function_t f)
{
    double h = (x1-x0)/n;
    double result=0.f;
    #pragma omp parallel for schedule(static) reduction(+:result)
    for (int i=0;i<n;++i) {
        double x = x0 + (i+.5f)*h;
        result += h*f(x);
    }

    return result;
}
```

4.8 Compiling and running an OpenMP application

We have a source file called *openmp.c*, and we want to compile this with OpenMP directives enabled. On a standard Linux computer this can be achieved by using the `-fopenmp` directive, i.e.

```
gcc -O3 -o openmp -fopenmp -c openmp.c
```

while using the Intel compiler as we do on Kongull or Vilje, the directive is simply `-openmp`, i.e.

```
icc -O3 -o poisson -openmp -c poisson.c
```

As long as we do not use any OpenMP utility function calls, we can still compile this code into a completely serial code, simply by removing the compiler switches. The compiler then simply ignore the pragmas, which makes the code look exactly as the serial code from its point of view. This is in stark contrast to a MPI version of the code where we would have to do substantial changes which makes the program dependent on the MPI libraries.

To run our program using for instance four threads we do

```
OMP_NUM_THREADS=4 ./openmp 2048
```

4.9 Final remarks

Modern supercomputers typically consist of multiple SMP nodes interconnected in a NUMA organization, see the lecture notes. Clusters also fall into the same category, since even the commodity processors used here have several cores integrated in their chips as discussed in the introduction. In particular, Vilje is an example of such a machine. Here each SMP node consists of 16 hyper-threaded processor cores. This means that an application which is parallelized using OpenMP can at most use 32 threads, although for floating point dominated programs, running only one thread per physical core is advised. If more computing resources are needed, we have no choice but to resort to a distributed memory approach using MPI. The same applies to Kongull, except here each SMP has 12 cores, which are not hyperthreaded.

A very natural question to ask is whether or not the two approaches can be combined. The answer to this is yes. In fact this approach often allows us the best of both worlds. Fine-grain parallelism is often intricate to exploit using a distributed memory model, while the convenience offered by the shared memory model often makes it fairly trivial to express. In addition, it is not always easy to say up front where exploiting fine-grained parallelism actually will improve the performance of your program. Since the modifications to the program using the OpenMP approach is minimal, we do not have to invest much effort just to benchmark whether or not parallelizing a particular loop improves performance.

Coarse-grain parallelism, however, is often fairly involved to exploit using a shared memory model, in particular due to the complications involved in protecting shared resources such as working buffers. This often leads to excessive memory usage or serialization of substantial parts of the code through usage of critical sections, see [subsection 4.5.1](#). Using a distributed memory model, this problem is nonexistent. Each process has their own private resources which are inaccessible from the other processes. Here expressing coarse grain parallelism is often just a matter of adjusting the limits on some loops, as well as adding the appropriate library calls for data exchanges between the processes when such exchanges are needed.

Hence a program where we utilize a message passing based approach, i.e. MPI, to express the coarse grain parallelism, while utilizing OpenMP pragmas to express the fine grain parallelism within each MPI process allows us to use each approach for what they are best at, while avoiding their weak sides.

4.10 Further reading

You can find the official OpenMP homepage at <http://openmp.org>. This page is a great resource for those who are interested in more details. In addition to having the description of the standard, it also contains links to several books on subject, as well as discussion forums where you can ask questions. If a source of tutorials are to be suggested, we can recommend <https://computing.llnl.gov/tutorials/openMP/>.